IDE agent workflows

Plan, act, observe, and review

The agent loop

```
Plan → Act → Observe → Review → Revise or stop
```

Use the loop to review the work.

Task contract

```
## Goal
Add one small, testable behavior.

## Constraints
- Named files and data only
- Required tests
- No new dependencies

## Done
- Tests pass
- A human accepts the change
```

Review checkpoints

- Read the proposed file changes.
- Read every command before it runs.
- Check test results and side effects.
- Stop when scope, data, or the stop rule is unclear.

Agenda

Time	Topic
0:00	Assistant-to-agent shift
0:08	Agent loop and tools
0:18	Task contracts and trust
0:28	Multi-file live demo
0:43	Chat vs. agent mode
0:50	Redirect and stop
0:57	Lab handoff

From autocomplete to agent

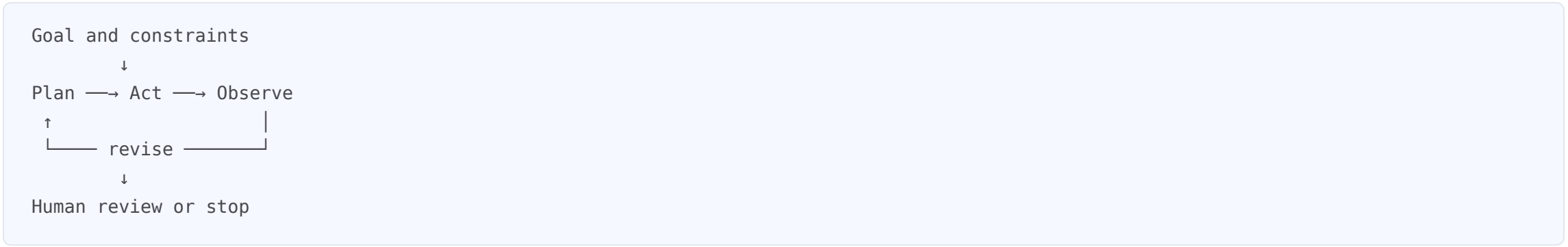
Mode	Developer supplies	Surface returns
Inline	Local code context	A continuation
Chat	A question and references	Guidance or a proposed edit
Agent	Goal, constraints, and repository	A multi-step change with a diff and check results

More autonomy needs clear file limits, progress checks, and review.

What agent mode can do

- inspect repository files and conventions;
- plan a sequence of changes;
- edit multiple related files;
- run terminal commands and tests;
- observe failures and revise;
- use tools, including configured MCP tools.

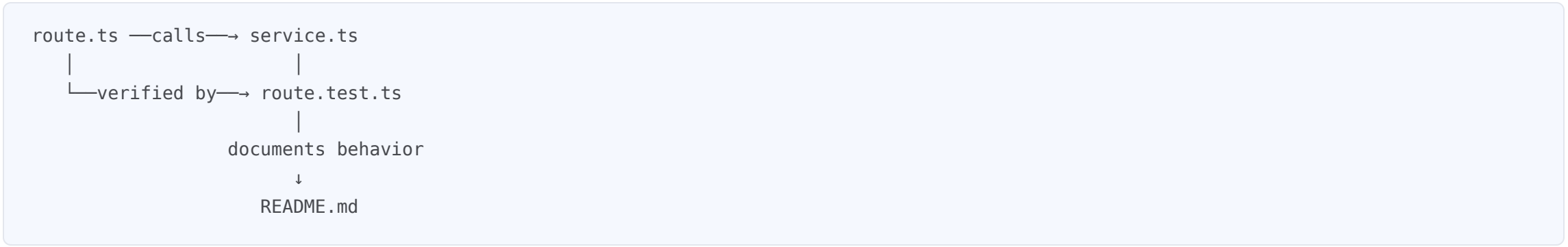
One loop, many checkpoints



Tool use and trust

Tool action	Review question
Read a file	Do you need this path and its data?
Edit a file	Does the task require it?
Run a command	Is it understood, reversible, and scoped?
Add a dependency	Is it necessary?
Search the web	Can repository context leave your environment?
Call an MCP tool	Which server, and what can it do?

Multi-file work is a graph



Name the behavior you want to change. Review every changed file and check that its callers and tests still fit.

Demo brief: one feature, three files

```
Add `completed` filtering to the task list endpoint.

Acceptance criteria:
- `GET /tasks?completed=true` returns completed tasks only.
- Invalid values return HTTP 400.
- Existing behavior without the parameter is unchanged.
- Focused tests cover true, false, and invalid values.

Constraints: no new dependencies; edit route, service, and tests only.
```

Concrete demo code

```
const value = req.query.completed;
if (value !== undefined && value !== "true" && value !== "false") {
  return res.status(400).json({ error: "completed must be true or false" });
}

const completed = value === undefined ? undefined : value === "true";
return res.json(taskService.list({ completed }));
```

Ask the agent to implement and test the smallest valid change, then explain it.

Narrate the demo

1. Show a clean working tree and the focused test command.
2. Paste the task contract and request a plan before edits.
3. Check that the plan names route, service, and tests.
4. Approve only commands needed for the task.
5. When a test fails, ask the agent to explain before revising.
6. Read the final diff and run tests yourself.

Intervene, redirect, stop

Use a correction that preserves the original contract:

```
Stop. Do not change the response schema or add dependencies.
Keep the existing service API and add only the optional filter.
Show the revised plan before editing.
```

Stop for unexpected file scope, destructive commands, sensitive data, unexpected network access, or repeated changes with no new test results.

Same task: Chat or agent mode?

Use Chat when...	Use agent mode when...
You need explanation or options	The outcome spans related files
You want one surgical edit	Tests can drive iteration
Requirements are still forming	Requirements and constraints are clear
You need to retain manual control	Tool output helps you check the work

Work manually when risk or ambiguity makes an agent a poor fit.

TODO code actions

A TODO can define a short task to delegate:

```
// TODO: reject duplicate task titles within the same project.
```

Before delegating:

- convert the comment into acceptance criteria;
- check the target repository and branch;
- remove sensitive context;
- define the tests.

Demo debrief

- Which plan step gave the earliest useful signal?
- Which command needed the most scrutiny?
- Did the agent touch exactly the expected files?
- Would Chat have been faster for this task?

Lab handoff

Build one Todo API, add ownership and due dates, then redirect a repository refactor. Along the way you will:

1. watch tool calls and multi-file edits;
2. redirect the agent after a scope change;
3. compare Ask and Agent mode;
4. try a TODO delegation if your account supports it.

Final deliverable: The tested API, plan, comparison, course correction, and your accept, revise, or stop decision.

Remember

1. Agent mode fits defined, testable, multi-step work.
2. Write the task limits before the agent starts.
3. Tool approval and file scope are trust decisions.
4. Redirect or stop the agent when needed.
5. You decide whether to accept the work.